NetPulse — Prototipo 1

Requisitos, diseño y API segura sobre Odoo (enfoque low-code)

Ingeniería de Software II (2016702) · Universidad Nacional de Colombia · Sede Bogotá · 2026-II

Integrantes: Juliana Parra Caro · Manuel Federico Castro Suarez · Fabian David Mora Martinez · Andres Felipe Arias Gonzalez

Entrega y sustentación: miércoles 11 de noviembre de 2026

1. Visión del producto y adaptación al enfoque low-code

Visión. En muchas redes institucionales el monitoreo (qué equipo cayó) y la gestión de incidencias (quién lo arregla) viven separados: alguien ve una alarma, abre un ticket a mano y el tiempo de respuesta depende de ese paso humano. **NetPulse** conecta ambos mundos: el simulador de red GNS3 reporta el estado de routers y switches, y Odoo registra, prioriza, asigna y hace seguimiento de las incidencias, creando automáticamente el ticket cuando un dispositivo crítico falla. Los usuarios son administradores de red y técnicos de soporte IT; el valor es reducir el tiempo entre la falla y la atención, y dejar trazabilidad completa.

Adaptación. Nuestro proyecto se construye sobre Odoo (plataforma low-code con ORM, autenticación, roles, interfaz web responsive, chatter y API externa ya incluidos). Por eso la guía del Prototipo 1, pensada para backends escritos desde cero, se cubre así: lo que Odoo ya resuelve se **configura y se documenta**; lo que no existe en Odoo se **adapta** con el equivalente más cercano; y lo que no aplica se **justifica**. La matriz completa está en el Anexo A.

Requisito de la guía	Cómo lo cubrimos en Odoo
Backend Spring Boot con CRUD	Modelos Odoo `netpulse.device` y `netpulse.incident`; el ORM entrega CRUD en UI y en API externa
JWT + registro/login	Autenticación nativa de Odoo (sesión y API keys con hash); JWT opcional con módulo OCA `auth_jwt`

Requisito de la guía	Cómo lo cubrimos en Odoo
Roles, 401 y 403	Grupos de seguridad, ACL y reglas de registro; `AccessDenied` (≈401) y `AccessError` (≈403)
OpenAPI 3	`openapi.yaml` escrito para las operaciones JSON-RPC del MVP (alternativa: OCA `base_rest`)
Secretos fuera del código	Variables de entorno en Docker (`.env` fuera del repositorio)
Postman	Colección sobre `/jsonrpc` con pruebas de login, CRUD, acceso sin credencial, rol insuficiente y datos inválidos

2. Requisitos

2.1 Roles de usuario

Rol (grupo Odoo)	Permisos
NetPulse Administrador	CRUD completo de dispositivos e incidencias; asigna técnicos y prioridades; ve dashboard y topología; gestiona usuarios
NetPulse Técnico	Lee dispositivos; ve y edita solo sus incidencias asignadas; cambia el estado de sus incidencias; no elimina ni reasigna
Sistema (usuario técnico del monitor)	Solo actualiza estado de dispositivos y crea incidencias automáticas

2.2 Historias de usuario y criterios de aceptación

HU-01 Como administrador quiero ver un dashboard con dispositivos activos y caídos para detectar de un vistazo si la red está en riesgo.

- Dado que inicié sesión como administrador y hay dispositivos registrados, cuando abro el dashboard, entonces veo contadores de activos y caídos, y los críticos caídos aparecen en el bloque superior.

- Dado que no he iniciado sesión, cuando abro la URL del dashboard, entonces Odoo me redirige al login.

HU-02 Como administrador quiero ver la topología con indicadores verde/rojo para ubicar rápidamente la falla.

- Dado que hay dispositivos con estado conocido, cuando abro la topología, entonces cada dispositivo es verde si está activo y rojo si está caído.
- Dado que GNS3 no responde, cuando abro la topología, entonces veo la fecha de la última actualización y un aviso de datos desactualizados.

HU-03 Como administrador quiero que se cree un ticket automáticamente cuando un dispositivo crítico cae para no depender del reporte manual.

- Dado que un dispositivo crítico pasa a caído y no tiene incidencia abierta, cuando corre el ciclo del monitor, entonces se crea una incidencia en estado Pendiente, prioridad Alta y causa raíz (ej. “SW2 offline”).
- Dado que ya existe una incidencia abierta para ese dispositivo, cuando el monitor lo detecta caído otra vez, entonces no se crea un duplicado y se registra una nota en la existente.

HU-04 Como técnico quiero ver y filtrar la bandeja de incidencias por estado y prioridad para atender primero lo urgente.

- Dado que hay incidencias de varias prioridades, cuando filtro por “Alta”, entonces solo veo esas.
- Dado que mi usuario no pertenece a ningún grupo NetPulse, cuando intento abrir el menú o el modelo, entonces el menú no aparece y la API responde AccessError.

HU-05 Como administrador quiero asignar un técnico a una incidencia para que alguien sea responsable de resolverla.

- Dado que hay una incidencia sin técnico, cuando la asigno, entonces queda registrada y el técnico recibe una actividad pendiente.
- Dado que soy técnico, cuando intento reasignar una incidencia, entonces el campo es de solo lectura y la API responde AccessError.

HU-06 Como administrador quiero cambiar la prioridad de una incidencia para reflejar su impacto real.

- Dado que una incidencia es Media, cuando la cambio a Alta, entonces el cambio queda en el historial (chatter) con usuario y fecha.

- Dado que soy técnico, cuando intento cambiar la prioridad, entonces el sistema lo rechaza con AccessError.

HU-07 Como técnico quiero ver solo mis incidencias asignadas para enfocarme en mi trabajo.

- Dado que tengo incidencias asignadas, cuando abro “Mis incidencias”, entonces solo veo las mías.
- Dado que otra persona tiene una incidencia, cuando intento abrirla por su identificador, entonces Odoo responde AccessError (regla de registro).

HU-08 Como técnico quiero ver el detalle de una incidencia con la causa raíz arriba para diagnosticar sin desplazarme.

- Dado que abro una incidencia, cuando carga el formulario, entonces dispositivo, causa raíz, prioridad y estado se ven sin hacer scroll.
- Dado que la incidencia no existe o fue eliminada, cuando intento abrirla, entonces veo un mensaje de error controlado, sin traza interna.

HU-09 Como técnico quiero cambiar el estado (Pendiente → En proceso → Resuelta) para reflejar el avance.

- Dado que la incidencia está En proceso, cuando la marco Resuelta, entonces se guarda la fecha de cierre.
- Dado que está Pendiente, cuando intento pasarla directo a Resuelta, entonces el sistema rechaza la transición con ValidationError.

HU-10 Como técnico quiero crear una incidencia manual para registrar fallas que el monitoreo no detecta.

- Dado que completo dispositivo y descripción, cuando guardo, entonces se crea con origen “Manual” en estado Pendiente.
- Dado que no selecciono dispositivo, cuando guardo, entonces el sistema responde con error de campo obligatorio (equivalente a 400).

HU-11 Como administrador quiero registrar dispositivos con tipo, criticidad e identificador GNS3 para que el monitor sepa qué vigilar.

- Dado que ingreso nombre, tipo, criticidad y nodo GNS3 válidos, cuando guardo, entonces el dispositivo aparece en el inventario.
- Dado que soy técnico, cuando intento eliminar un dispositivo, entonces el sistema responde AccessError.

HU-12 Como administrador quiero asignar roles a los usuarios para controlar quién puede hacer qué.

- Dado que creo un usuario y le asigno el grupo Técnico, cuando inicia sesión, entonces solo ve las opciones de ese rol.
- Dado que ingreso credenciales incorrectas, cuando intento iniciar sesión, entonces el acceso es denegado sin revelar si falló el usuario o la contraseña.

2.3 Story map y MVP

Actividad	Versión 1 (MVP)	Versión 2	Versión 3
Administrar inventario y accesos	HU-11 Dispositivos, HU-12 Roles		Importar topología desde GNS3
Monitorear la red	HU-01 Dashboard, HU-02 Topología	Gráficas históricas de disponibilidad	Mapa interactivo (widget OWL)
Generar incidencias	HU-03 Ticket automático	HU-10 Ticket manual	Correlación de fallas en cascada
Gestionar incidencias	HU-04 Bandeja, HU-05 Asignar, HU-08 Detalle	HU-06 Prioridad	SLA y reportes
Resolver incidencias	HU-07 Mis incidencias, HU-09 Estados		Notificaciones por correo

MVP: HU-01, 02, 03, 04, 05, 07, 08, 09, 11 y 12.

2.4 Estimación (Planning Poker, Fibonacci)

HU	Puntos	HU	Puntos
HU-01	3	HU-08	3
HU-02	5	HU-09	3
HU-03	8	HU-11	3
HU-04	3	HU-12	1
HU-05	2	HU-07	2

Total del MVP: **33 puntos**. HU-03 es la más incierta (integración con GNS3 y deduplicación). El tablero de GitHub Projects tiene una tarjeta por historia del MVP con su estimación: [ENLACE AL TABLERO].

3. Experiencia de usuario

Wireframes. Los wireframes de baja y alta fidelidad (Dashboard, Topología, Bandeja, Detalle de incidencia y Cierre) están en Figma, usando la paleta y tipografía del wireframe oficial de Odoo: https://www.figma.com/design/Z7XpwDRNMefjx80Mwmpafs/Wireframe-NetPulse?node-id=1-8401&t=YOqq5XuvQFGRLyLy-1 (verificar que el enlace esté en modo de solo lectura).

Flujo de navegación. Login → Dashboard → Topología → Bandeja de incidencias → Detalle de incidencia → Cierre (Pendiente → En proceso → Resuelta). El técnico entra por “Mis incidencias” y el administrador por el Dashboard.

Modelo de cliente: web responsive sobre el cliente web de Odoo. Los operadores necesitan paneles amplios, y Odoo ya entrega un cliente web responsive (listas, kanban, formularios) que funciona también en el navegador del celular, sin desarrollar ni mantener un cliente aparte. Descartamos app nativa o híbrida porque el costo de mantenimiento no se justifica para un uso principalmente de escritorio. **Versión móvil:** no se diseña un wireframe separado; el diseño responsive de Odoo cubre la vista del técnico en campo (bandeja y detalle). Si el profesor lo exige, se agrega un wireframe móvil de esas dos pantallas.

Brecha entre el diseño y Odoo. Dashboard, bandeja y detalle se construyen con vistas nativas (kanban, lista, formulario, gráfico). La topología tipo mapa interactivo no existe de forma nativa: en el MVP se representa como un kanban de dispositivos agrupado por tipo

con tarjetas verdes/rojas, y el mapa interactivo queda para la Versión 3.

4. Diseño de la solución

4.1 Arquitectura

Componente	Tecnología	Estilo arquitectónico y justificación
Cliente web	Cliente web de Odoo (OWL, responsive)	Cliente delgado servido por el propio servidor; sin cliente móvil separado
Backend	Odoo 17 Community (Python)	**MVC** propio de Odoo: modelos (ORM), vistas (XML) y controladores/acciones; adoptado porque Odoo lo impone y reduce el código
Servicio de monitoreo	Acción programada (cron) de Odoo que consulta la API REST de GNS3	**Adaptador** (ports and adapters aplicado solo a esta integración): aísla el formato de GNS3 del dominio
Base de datos	PostgreSQL 15	Persistencia gestionada por el ORM
Simulador	GNS3 Server	Sistema externo (contexto Monitoreo)

Flujo: GNS3 → (HTTP, API REST) → Adaptador en Odoo → Dominio de incidencias → Cliente web. Odoo además expone su API externa (JSON-RPC) para pruebas y futuras integraciones.

4.2 Modelo de dominio (DDD)

Glosario del lenguaje ubicuo

Término	Significado
Dispositivo	Router o switch registrado, ligado a un nodo de GNS3
Criticidad	Importancia del dispositivo para la red (Crítico / Normal)

Término	Significado
Estado del dispositivo	Activo, Caído o Desconocido
Falla crítica	Cambio a Caído de un dispositivo con criticidad Crítico
Incidencia (ticket)	Registro de una falla que debe atenderse
Causa raíz	Descripción técnica del origen de la falla (ej. “SW2 offline”)
Prioridad	Urgencia de atención: Baja, Media, Alta
Estado de la incidencia	Pendiente, En proceso o Resuelta
Técnico	Usuario responsable de resolver la incidencia
Origen	Automático (monitor) o Manual
Topología	Conjunto de dispositivos y su estado visible
Ciclo de monitoreo	Una consulta periódica a GNS3

Bounded contexts y context map

- **Monitoreo:** dispositivos, estado, consulta a GNS3.
- **Gestión de incidencias:** incidencias, prioridades, asignación y estados.
- **Identidad y acceso:** usuarios, grupos y permisos (provistos por Odoo).

Relaciones: Monitoreo es *upstream* de Gestión de incidencias (cliente/proveedor): publica el hecho “dispositivo crítico caído”. El **adaptador GNS3** actúa como capa anticorrupción, de modo que el vocabulario de GNS3 (`node` , `started` , `stopped`) no entra al dominio. Identidad y acceso es un *conformist* de Odoo (adoptamos su modelo tal cual).

Agregado Incidencia

- **Raíz:** `Incidencia` .
- **Entidades:** entradas de seguimiento (mensajes y cambios del chatter).
- **Value objects:** Prioridad, EstadoIncidencia, CausaRaíz.
- **Invariantes:** (1) solo una incidencia abierta por dispositivo; (2) las transiciones válidas son Pendiente → En proceso → Resuelta; (3) una incidencia En proceso o Resuelta tiene técnico asignado; (4) una incidencia Resuelta tiene fecha de cierre y no se reabre.

4.3 Diagrama de clases (dominio y aplicación) y SOLID

Clase (modelo Odoo)	Atributos principales	Métodos / responsabilidad
`netpulse.device`	name, gns3_node_id, device_type, criticality, status, last_seen	`update_status(is_up)`: actualiza estado y dispara la falla crítica
`netpulse.incident` (raíz)	name, device_id, priority, state, technician_id, root_cause, origin, date_detected, date_resolved	`create_from_monitor(device)`, `write()` valida transiciones, `action_start()`, `action_resolve()`
`netpulse.gns3.adapter`	base_url, project_id	`fetch_nodes()`: traduce nodos GNS3 a estados de dispositivo
`res.users` / `res.groups` (Odoo)	grupos Administrador y Técnico	Autorización

Relaciones: un Dispositivo tiene muchas Incidencias (solo una abierta); un Técnico (`res.users`) tiene muchas Incidencias.

- **S (responsabilidad única):** el adaptador solo traduce GNS3; el dispositivo solo gestiona su estado; la incidencia solo gestiona su ciclo de vida.
- **O (abierto/cerrado):** otro tipo de fuente de monitoreo se agrega implementando otro adaptador, sin tocar la incidencia.
- **D (inversión de dependencias):** el dominio recibe estados desde el adaptador; no conoce el formato de GNS3.

4.4 Patrones de diseño (GoF)

Patrón	Problema real que resuelve	Participantes
Adapter	GNS3 usa su propio vocabulario y formato JSON; el dominio no debe depender de él	Target: `update_status(is_up)`; Adaptee: API REST de GNS3; Adapter: `netpulse.gns3.adapter`

Patrón	Problema real que resuelve	Participantes
State	Las incidencias tienen transiciones válidas limitadas y reglas distintas por estado	Context: `netpulse.incident`; estados: Pendiente, En proceso, Resuelta; tabla de transiciones permitidas en `write()`

Además, las acciones automáticas de Odoo (notificar al asignar, registrar fecha de cierre) siguen el patrón **Observer** que el propio framework implementa.

4.5 Atributos de calidad

Escenario 1 — Seguridad

Parte	Descripción
Fuente	Usuario autenticado con rol Técnico
Estímulo	Intenta eliminar un dispositivo o abrir la incidencia de otro técnico
Artefacto	Modelos `netpulse.device` y `netpulse.incident`
Entorno	Operación normal
Respuesta	Se deniega con AccessError y queda en el log del servidor
Medida	100 % de los intentos denegados; 0 datos expuestos

Táctica: autorizar usuarios (ACL por grupo y reglas de registro).

Escenario 2 — Rendimiento

Parte	Descripción
Fuente	Monitor (cron de Odoo)
Estímulo	Un dispositivo crítico pasa a Caído
Artefacto	Servicio de monitoreo y modelo de incidencias
Entorno	Operación normal, hasta 20 dispositivos
Respuesta	Se crea la incidencia automática con causa raíz

Parte	Descripción
Medida	En máximo 2 minutos desde la caída (intervalo de 1 minuto)

Táctica: gestionar recursos con consulta periódica y procesamiento en un solo ciclo.

Escenario 3 — Disponibilidad / robustez

Parte	Descripción
Fuente	Servidor GNS3
Estímulo	No responde (timeout)
Artefacto	Adaptador GNS3
Entorno	Operación degradada
Respuesta	El ciclo registra el error, marca los estados como Desconocido y no crea tickets falsos; reintenta en el siguiente ciclo
Medida	0 incidencias falsas; recuperación automática en el siguiente ciclo

Táctica: detección de fallas (timeout y manejo de excepciones) y reintento.

5. API segura (adaptación a Odoo)

5.1 Recursos y CRUD. `netpulse.device` y `netpulse.incident` tienen CRUD completo mediante la UI y mediante la API externa de Odoo (`create`, `read` / `search_read`, `write`, `unlink` vía `execute_kw`).

5.2 Autenticación. Odoo autentica con usuario y contraseña (sesión) o con *API keys* para integraciones; ambas se almacenan con hash, nunca en texto plano. El equivalente a "registro" es la creación de usuarios por el administrador (HU-12). Dejamos explícito que **no hay JWT nativo**; si se exige, se instala el módulo OCA `auth_jwt`.

5.3 Autorización. Archivos `ir.model.access.csv` (permisos por modelo y grupo) y reglas de registro (el técnico ve solo `technician_id = usuario`). La API no devuelve 401/403 HTTP sino HTTP 200 con un cuerpo de error: `AccessDenied` (credenciales inválidas, ≈401) y `AccessError` (sin permiso, ≈403); la documentación y las pruebas lo indican así.

5.4 Buenas prácticas. Credenciales de PostgreSQL, contraseña maestra de Odoo y URL/proyecto de GNS3 en variables de entorno (`.env` en `.gitignore` , `.env.example` en el repositorio); administrador de bases de datos deshabilitado (`list_db = False`); validación con campos obligatorios, `@api.constrains` y `ValidationError` ; Odoo no expone trazas al cliente en producción (modo no desarrollador).

5.5 HTTPS. Odoo se despliega detrás de un proxy inverso (Caddy o Nginx con Let's Encrypt) con `proxy_mode = True` y redirección de HTTP a HTTPS. Si no se despliega antes de la entrega, se describe esta configuración y se demuestra en local.

5.6 OpenAPI. Odoo no genera OpenAPI. Entregamos `openapi.yaml` (OpenAPI 3.0, validado en Swagger Editor) con las operaciones del MVP sobre `/jsonrpc` (`authenticate` , `create` , `search_read` , `write` , `unlink` para dispositivos e incidencias), esquemas, ejemplos, errores y esquema de seguridad por API key.

5.7 Postman. Colección con pruebas automáticas: autenticación válida, CRUD de ambos recursos, autenticación inválida (`AccessDenied`), técnico intentando eliminar un dispositivo (`AccessError`) y datos inválidos (`ValidationError`). Se verifica el cuerpo (`json.error`) y no el código HTTP.

6. Guía de implementación en Odoo

Paso 1. Entorno. `docker-compose.yml` con `odoo:17` y `postgres:15` . Credenciales en `.env` . Instalar los módulos Discuss (`mail`) y Automatizaciones (`base_automation`).

Paso 2. Módulo `netpulse` . Carpeta `addons/netpulse` con `__manifest__.py` , `models/` , `security/` , `views/` y `data/` . Se versiona en GitHub, lo que da evidencia del proceso. Variante 100 % sin código: crear los modelos desde Ajustes → Técnico (modo desarrollador) o con Odoo Studio (Enterprise); en ese caso se documentan con capturas.

Paso 3. Modelo de incidencia (esqueleto).

```
class Incident(models.Model):
    _name = "netpulse.incident"
    _inherit = ["mail.thread", "mail.activity.mixin"]
    _TRANSITIONS = {"pending": {"in_progress"}, "in_progress": {"resolved"},
"resolved": set()}

    device_id = fields.Many2one("netpulse.device", required=True)
    state = fields.Selection([("pending", "Pendiente"), ("in_progress", "En
proceso"),
                              ("resolved", "Resuelta")], default="pending",
tracking=True)
    priority = fields.Selection([("0", "Baja"), ("1", "Media"), ("2",
"Alta")], tracking=True)
    technician_id = fields.Many2one("res.users", tracking=True)
    root_cause = fields.Char()

    def write(self, vals):
        if "state" in vals:
            for rec in self:
                if vals["state"] != rec.state and vals["state"] not in
self._TRANSITIONS[rec.state]:
                    raise ValidationError(_("Transición no permitida"))
        return super().write(vals)
```

El método `create_from_monitor(device)` busca una incidencia abierta del dispositivo; si existe, agrega una nota, y si no, la crea con prioridad Alta y causa raíz `"<dispositivo> offline"`.

Paso 4. Seguridad. Crear dos grupos (Administrador y Técnico) en `security/security.xml`; permisos en `ir.model.access.csv` (técnico: lectura de dispositivos, lectura y escritura de incidencias, sin eliminar); y una regla de registro para que el técnico solo vea sus incidencias.

Paso 5. Monitor con acción programada (ir.cron).

```
@api.model
def cron_sync_gns3(self):
    base, project = os.environ["GNS3_URL"], os.environ["GNS3_PROJECT_ID"]
    try:
        nodes = requests.get(f"{base}/v2/projects/{project}/nodes",
timeout=10).json()
    except requests.RequestException:
        return self.search([]).write({"status": "unknown"})
    for node in nodes:
        dev = self.search([("gns3_node_id", "=", node["node_id"])], limit=1)
        if dev:
            dev.update_status(node["status"] == "started")
```

`update_status` guarda el estado y `last_seen`; si el dispositivo es crítico y pasó a Caído, llama a `create_from_monitor`. Si Odoo corre en la nube y GNS3 en el computador local, no se alcanzan: en ese caso un script en la máquina de GNS3 hace el mismo ciclo y escribe en Odoo por la API externa con una API key.

Paso 6. Vistas. Kanban de dispositivos agrupado por tipo con tarjetas verde/rojo (topología); lista y kanban de incidencias con filtros (bandeja); formulario con causa raíz, dispositivo, prioridad y estado arriba y barra de estados (detalle); y vistas de gráfico y pivote para el dashboard (activos vs. caídos, incidencias por prioridad).

Paso 7. Automatizaciones. Acción automática al asignar técnico (crea una actividad al técnico) y al pasar a Resuelta (registra `date_resolved`).

Paso 8. Datos de demostración. Dos usuarios (admin y técnico), 5 a 6 dispositivos que correspondan a nodos reales de la topología de GNS3, y una incidencia de ejemplo.

Paso 9. Pruebas y despliegue. Colección Postman exportada a `/postman`, `openapi.yaml` a `/docs`, y proxy Caddy o Nginx para HTTPS.

7. Repositorio y trabajo en equipo

Estructura: `addons/netpulse/`, `docs/openapi.yaml`, `postman/`, `docker-compose.yml`, `.env.example`, `.gitignore` (incluye `.env` y datos de PostgreSQL) y `README.md` (descripción, integrantes, tecnologías, ejecución local, variables de entorno de ejemplo, enlaces a wireframes, API y tablero). Trabajo con ramas por historia, *pull requests* revisados por otro integrante y commits descriptivos de todos. Acceso otorgado a sevargasp@unal.edu.co.

8. Decisiones, riesgos y próximos pasos

Decisión o riesgo	Manejo
Odoo no genera OpenAPI ni emite JWT	Documentación manual y alternativa OCA; acordar con el profesor el criterio de evaluación
La topología interactiva no es nativa	MVP con kanban verde/rojo; widget OWL en Versión 3
Odoo en la nube no alcanza GNS3 local	Demo en local o script externo con API key
Errores de la API con HTTP 200	Pruebas sobre el cuerpo del error
Enterprise (Helpdesk, Studio) no disponible en Community	Módulo propio `netpulse`

Hacia el Prototipo 2: wizard de importación de topología, historial de disponibilidad con gráficas, notificaciones por correo, SLA por prioridad, y evaluación de `auth_jwt` y `base_rest` para una API REST formal.

Anexo A. Matriz de cobertura frente a la guía del Prototipo 1

Sección de la guía	Estado	Nota
1. Requisitos (a–f)	Cumple	Sección 2
2.a Wireframes web y móvil	Parcial	Web en Figma; móvil cubierto por el diseño responsive de Odoo
2.b Flujo de navegación	Cumple	Sección 3
2.c Modelo de cliente	Cumple	Web responsive sobre Odoo
3.a Arquitectura	Adaptado	MVC de Odoo + adaptador; no hay cliente móvil separado
3.b a 3.e Dominio, clases, patrones, calidad	Cumple	Sección 4
4.a OpenAPI	Adaptado	Escrito para la API externa de Odoo

Sección de la guía	Estado	Nota
4.b CRUD en dos recursos	Cumple	Vía ORM
4.c JWT y hash	Adaptado	API keys y sesión con hash; JWT opcional
4.d Roles, 401/403	Adaptado	AccessDenied y AccessError
4.e Buenas prácticas	Cumple	Variables de entorno, validación, errores controlados
4.f HTTPS	Cumple o describe	Proxy con TLS
4.g Postman	Cumple	Sobre `/jsonrpc`
4.h API externa (bono)	Cumple de forma natural	GNS3 se consume por un adaptador; se puede reclamar como bonificación si el profesor lo acepta
5. Repositorio	Cumple	Sección 7